Deep Dive Technical Documentation of pgBackRest: Architecture, Internal Implementation, and Security

1. Introduction and Architectural Overview

In the landscape of database disaster recovery, pgBackRest has established itself as an engineering-focused, high-performance backup and restore solution constructed specifically for PostgreSQL environments. Originally developed in Perl, the software underwent a comprehensive migration to a single-executable C application to maximize performance, eliminate dependencies on external runtime environments, and deeply integrate with PostgreSQL's internal data formats[1]. By discarding reliance on traditional, generic UNIX utilities such as tar or rsync, pgBackRest natively addresses database-specific challenges, including page-level checksum validation, efficient Write-Ahead Log (WAL) archiving, and multi-threaded stream compression[2].

The application operates on the foundational concept of a "stanza," a central organizational unit representing a single PostgreSQL cluster[2]. The stanza maps the physical database paths to the corresponding backup repositories, enabling pgBackRest to manage backups, continuous WAL archiving, and restoration through a unified configuration typically defined in /etc/pgbackrest/pgbackrest.conf[4]. This design simplifies multi-repository architectures, allowing administrators to maintain a local repository for rapid, short-term recoveries alongside a remote object storage repository for long-term compliance retention[6].

The primary architectural differentiator of pgBackRest lies in its execution and communication model. It functions via a custom internal Inter-Process Communication (IPC) protocol, facilitating seamless local or remote operations over SSH or TLS[6]. The tool runs strictly on the database host or a dedicated repository host. It does not require direct SQL-level access to the PostgreSQL database cluster to perform its core storage duties, drastically reducing the attack surface and isolating the database engine from the heavy I/O overhead of compression and encryption[6].

2. Core Subsystems and C Implementation Mechanics

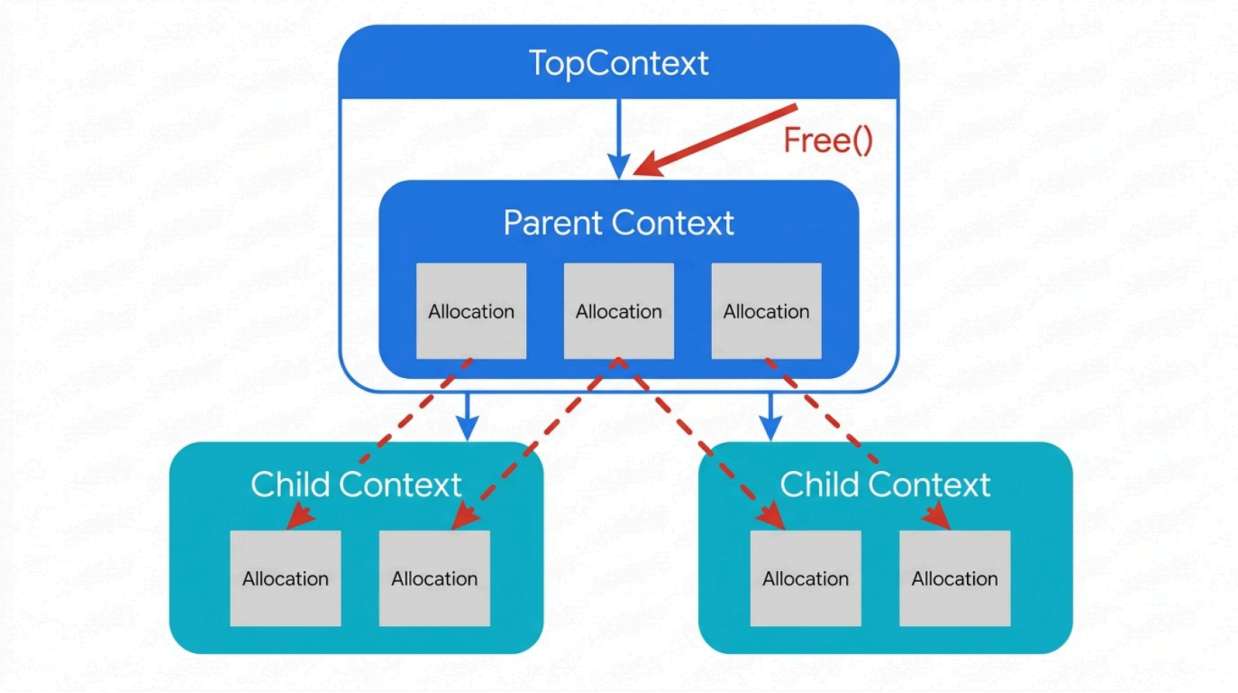
The complete migration of pgBackRest to C required the development of sophisticated, internal subsystems to manage dynamic memory, handle runtime exceptions, and coordinate parallel I/O streams safely[1]. To achieve the safety of a high-level managed language while maintaining the raw processing speed of bare-metal C, the project maintainers engineered a custom hierarchical memory management framework and a macro-driven exception-handling system.

2.1 Memory Management: The MemContext Paradigm

In standard C application development, manual memory allocation via malloc and free is highly prone to memory leaks, particularly in complex applications featuring extensive asynchronous processes and multiple failure pathways. pgBackRest resolves this architectural vulnerability by implementing a hierarchical memory context system, heavily inspired by PostgreSQL's own internal MemoryContext design, which is managed within src/common/memContext.h and src/common/memContext.c[8].

The MemContext structure organizes memory allocations into a deterministic tree. When a parent context is freed or discarded, all of its nested child contexts and associated memory blocks are automatically destroyed. This hierarchical garbage collection ensures that temporary data structures spawned during complex operations—such as parsing WAL file headers, processing JSON responses, or building an extensive backup manifest—are reliably cleared from memory without requiring exhaustive tracking by the developer[8].

Hierarchical Memory Context Architecture



The MemContext system prevents memory leaks by linking allocations to hierarchical contexts. Discarding a parent context automatically frees all nested child contexts and raw allocations.

The underlying mechanics of the MemContext system utilize aggressive optimization strategies to minimize overhead. Space is not prematurely reserved for child contexts, as most contexts function as leaves with no children. When a child context is eventually requested, space for a default number of child contexts (MEM_CONTEXT_INITIAL_SIZE, strictly set to 4) is allocated[8]. If the allocation limit is exceeded as more contexts are added, the allocated space doubles

dynamically to accommodate the growth while preventing memory fragmentation[8].
Operations dynamically switch active contexts using the memContextSwitch(MemContext *this) function. Any subsequent calls to allocation functions, such as memNew(size_t size) or memResize(void *buffer, size_t size), are automatically mapped to the currently active context[8].
For highly iterative tasks, such as scanning thousands of WAL files, pgBackRest provides specialized macros. The macros MEM_CONTEXT_TEMP_BEGIN() and MEM_CONTEXT_TEMP_END() establish isolated scopes for transient allocations. To prevent memory bloat in deep loops, developers use MEM_CONTEXT_TEMP_RESET_BEGIN(), which allows the temporary context to cleanly reset after a predefined number of iterations[8].

2.2 Exception Handling: TRY, CATCH, and setjmp

Because the C language does not possess native object-oriented exception handling, pgBackRest constructs a robust try/catch mechanism utilizing the standard C library functions setjmp and longjmp. This logic is encapsulated within macros defined strictly in src/common/error/error.h[9].
The architecture relies on the orchestration of TRY_BEGIN(), CATCH(), FINALLY(), and TRY_END() macros. When a TRY_BEGIN() block is invoked, setjmp takes a snapshot of the current execution context, including the stack pointer, the instruction pointer, and the pointer to the currently active MemContext[8]. If an error condition occurs deeper in the execution stack, the THROW macro is executed, which invokes longjmp, instantly returning execution flow directly back to the setjmp checkpoint[8].
This macro-driven exception subsystem is inextricably linked with the memory management framework. If an error forces a sudden execution jump, the memory manager automatically discards any temporary memory contexts that were generated within the failed TRY block. Only contexts explicitly protected using the memContextKeep() function survive the automatic cleanup[8]. This synergy guarantees that volatile error conditions do not leave dangling pointers or cause compounding memory leaks.

2.3 The Custom IPC Protocol and Process Forking

pgBackRest achieves high-throughput data transfer and compression via a custom Inter-Process Communication (IPC) protocol, implemented across files in src/protocol/ (such as client.c, server.c, and protocol.c)[10]. Based on the --process-max configuration parameter, the main application forks multiple parallel worker processes to handle localized I/O, hashing, and compression[2].
The protocol operates strictly as a deterministic state machine, transitioning between discrete states such as idle, data-get, and response[10]. This approach requires high stream fidelity. A frequent diagnostic issue arises when administrators route protocol traffic over SSH instead of TLS. If the target server's SSH daemon injects unexpected text into the stream—such as Message of the Day (MOTD) banners, user broadcast warnings, or shell startup scripts—the protocol stream becomes corrupted. This disruption immediately triggers the documented ERROR [039]: client state is 'response' but expected 'idle' or ERROR [101][12]. These specific error codes rarely indicate a bug in the backup logic itself; rather, they signify that an underlying

system anomaly or network noise disrupted the IPC state machine[12]. For reliable enterprise deployments, the tool heavily favors TLS connections (tls-server-auth) to bypass SSH entirely, providing secure, noise-free, socket-based communication between the database and the repository host[10].

3. Configuration Architecture and Parameter Evaluation

The software provides a highly flexible configuration framework capable of mapping intricate, multi-node cluster topologies. While pgBackRest can be operated entirely via command-line arguments, leveraging the /etc/pgbackrest/pgbackrest.conf file is the standard practice for persistent configuration[5]. The configuration parser rigorously evaluates incoming parameters against strict type definitions to prevent runtime logic errors.

Parameter Type	Functional Description and Examples
String	Alphanumeric text typically used for passwords, hostnames, and specific identifiers. Example: repo1-cipher-pass=zWaf6XtpjIVZC...[5].
Path	Absolute filesystem locations. Paths must strictly begin with /, cannot contain a double slash //, and require no trailing slash. Example: repo1-path=/var/lib/pgbackrest[5].
Boolean	Binary toggles accepting only y or n. Example: start-fast=y[5].
Integer	Whole numbers utilized for defining network ports, retention counts, and maximum parallel processes. Example: compress-level=3[5].
Size	Defines storage buffers and queue limits. Specified in raw bytes by default, or with multipliers (KiB, MiB, GiB, TiB, PiB). Fractional values are rejected. Example: archive-get-queue-max=1GiB[5].
Time	Defines timeout constraints in seconds. Example: io-timeout=90[5].

List / Key-Value	Iterative options defined multiple times across multiple lines. Example: tablespace-map=ts_01=/db/ts_01[5].

The parsing engine respects a strict hierarchy of overrides. Any option defined in the configuration file can be overridden by environment variables (formatted with a PGBACKREST_ prefix, such as PGBACKREST_PG1_PATH), which can in turn be overridden by explicit command-line flags[15]. Furthermore, the system includes a reset- prefix for command-line options. This feature is particularly useful for overriding non-boolean configurations during edge-case recovery scenarios. For instance, an administrator can execute a restore directly on a remote repository host by passing --reset-pg1-host, instructing pgBackRest to ignore the configured remote database host parameter and force a local restore procedure[15].

4. Implementation of Key Database Actions

The pgBackRest architecture strictly segments its operational capabilities into distinct internal commands, primarily backup, restore, and archiving functions. Each action incorporates deep PostgreSQL logic to ensure data consistency.

4.1 Backup Operations (src/command/backup/backup.c)

The core engine responsible for data protection is executed via the backup command. It evaluates the current state of the repository and seamlessly executes one of several backup paradigms based on the requested --type[15].

Backup Type	Technical Mechanism
Full	Copies the entire contents of the $PGDATA cluster to the repository. It serves as an independent consistency baseline requiring no prior backups to restore[17].
Differential	Evaluates the manifest and copies only the database cluster files that have changed since the most recent successful Full backup[2].
Incremental (File-Level)	Copies only files modified since the *last* backup of any type (Full, Differential, or Incremental). Creates a dependent chain requiring all previous incrementals back to a reliable baseline[17].

Incremental (Block-Level)	Activated via --repo-block, the software calculates block-level deltas within large PostgreSQL relation files, transmitting only the changed bytes rather than the entire file[6].

Timestamp Resolution and Deterministic Timing

During file-level differential and incremental backups, pgBackRest relies heavily on file modification timestamps to dictate whether a file must be included in the backup stream. However, traditional POSIX filesystems frequently lack high-precision, sub-second timestamp resolution[21].

If PostgreSQL modifies a data file during the exact same second that a backup process reads the manifest and copies that file, a subsequent modification by PostgreSQL within that identical second will not result in a new timestamp[21]. Generic tools lacking database awareness (such as standard rsync without strict checksumming) would falsely conclude the file remained unchanged during the next incremental run, leading directly to data loss[21].

The backup.c implementation neutralizes this race condition utilizing a deterministic timing lock. After building the internal backup manifest, the process initiates a thread sleep function for the remainder of the current second before executing any file copy operations[21]. This deliberate delay ensures that any active modifications made by the database engine within that second are securely captured, and any modifications occurring afterward will accurately cross the threshold into a newer, detectable timestamp[21]. For environments requiring cryptographic certainty over speed, administrators can pass the --delta flag, forcing pgBackRest to ignore timestamps entirely and calculate cryptographic checksums for every block[4].

4.2 Restoration Procedures (src/command/restore/restore.c)

The restore command executes sophisticated recovery procedures, rebuilding the database cluster directly from the repository. It handles in-place directory replacements, restorations targeting completely new environments, and automated injection of Point-in-Time Recovery (PITR) targets directly into postgresql.auto.conf or recovery.conf files[4].

A critical feature defined within restore.c is the Delta Restore functionality, triggered by the --delta parameter. Rather than blindly wiping the existing $PGDATA directory and streaming the entire multi-terabyte dataset across the network, the delta restore logic cross-references the checksums of the files currently residing on the corrupted database host against the targeted backup manifest[4]. It subsequently pulls only the differing blocks from the repository. For large clusters suffering from minor corruption or slight data drift, a delta restore bypasses the network bottleneck and can reduce recovery times by upwards of 80%[4].

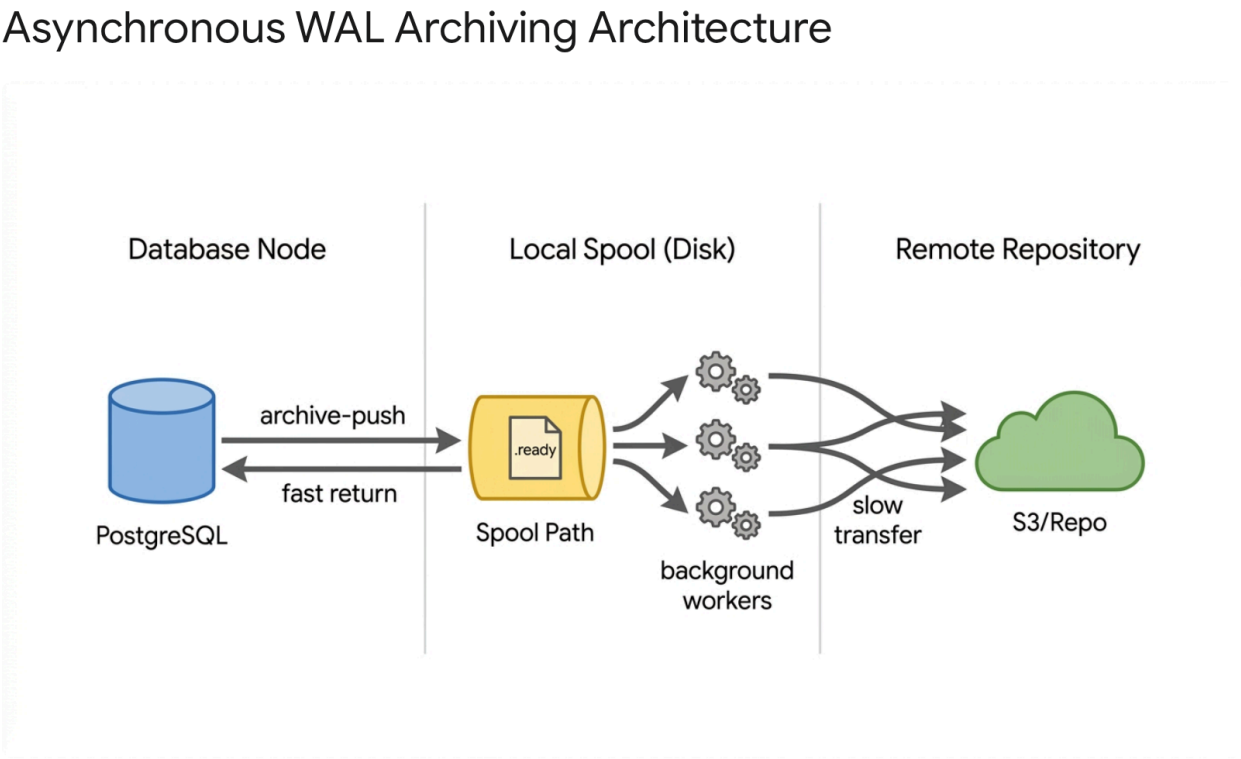
4.3 The Archiving Subsystem (archive-push and archive-get)

Continuous WAL archiving is the absolute backbone of a resilient PostgreSQL Point-in-Time Recovery strategy[2]. PostgreSQL delegates the handling of these critical 16MB log segments to pgBackRest via the archive_command setting in postgresql.conf[4].

Asynchronous Archiving Mechanics

Executing a standard, synchronous archive-push forces the primary PostgreSQL engine to pause, wait for the repository connection to initialize, compress the segment, and complete the network transfer before it can recycle the local WAL file[25]. Under severe transactional load, this latency causes WAL files to rapidly accumulate on the primary storage volume, potentially leading to disk exhaustion and a total database crash[5].

To isolate PostgreSQL from repository network latency, pgBackRest utilizes an asynchronous architecture (archive-async=y), relying on a dedicated local spool directory (spool-path)[5].



By enabling archive-async, pgBackRest decouples PostgreSQL from network latency. The archive_command returns instantly after writing to the local spool, while background workers manage the remote transfer.

When asynchronous archiving is enabled, the workflow shifts dramatically. PostgreSQL invokes archive-push, and pgBackRest drops the WAL file reference into the local spool-path queue, marking it with a .ready state file. Crucially, the foreground process immediately returns a successful exit code (0) to PostgreSQL[25]. PostgreSQL instantly drops its internal lock and proceeds with normal operations. Simultaneously, a persistent background pgBackRest worker process monitors the spool-path. Leveraging the assigned process-max cores, it compresses and transmits the queued .ready files to the repository in parallel[25]. Once successful, the flag is transitioned to .ok, signaling completion.

If the remote repository becomes inaccessible, WAL files will queue locally in the spool directory. The --archive-push-queue-max parameter provides a highly effective safety valve for this scenario. If the local queue reaches this threshold (for instance, configured to 10GB), pgBackRest forcibly stops archiving, silently drops the oldest WAL files, and returns success to PostgreSQL[5]. While this intentional data destruction severs the Point-in-Time Recovery chain and necessitates an immediate fresh Full backup, it fulfills its primary design directive: prioritizing the uptime of the production PostgreSQL database over the continuity of the backup archive by preventing catastrophic disk exhaustion[5].
During restoration, the --archive-missing-retry flag commands the archive-get process to aggressively retry fetching WAL segments that were previously deemed missing in an asynchronous queue, preventing premature recovery failures due to minor network delays and keeping PostgreSQL streaming properly from the archive[5].

5. Storage Drivers, I/O Filters, and Network Resilience

Data streaming in pgBackRest is governed by an internal I/O Filter framework (src/common/io/filter/filter.h). This subsystem permits data streams to be dynamically chained through compression algorithms (such as lz4, zstd, or gzip) and encryption layers entirely in-memory before the finalized byte stream is written to the physical storage socket[6].

5.1 Object Storage Dynamics and File Bundling

While local POSIX filesystems operate highly efficiently when managing millions of microscopic files, cloud object stores (such as AWS S3, Google Cloud Storage, or Azure Blob) incur massive latency penalties due to high per-object REST API request overheads[16]. A standard PostgreSQL backup may consist of tens of thousands of tiny relation files and zero-byte forks. Pushing these files individually to an S3 bucket causes backups to run exponentially slower than local I/O, while heavily increasing cloud transaction costs[16].
To optimize object storage interactions, pgBackRest relies on the repo-bundle parameter. This architecture concatenates multiple small database files into large, cohesive block bundles during the backup stream. Furthermore, zero-byte files are not pushed to storage at all; they are recorded purely as metadata in the manifest[16]. By drastically reducing the volume of discrete PUT requests, bundling maximizes network bandwidth utilization[16].
In high-security environments, administrators frequently utilize Object Lock features (e.g., S3 Compliance Mode) to defend against ransomware. Because these locks are strictly enforced by the cloud provider, pgBackRest's retention enforcement tool (the expire command) will encounter 403 Forbidden errors when attempting to rotate out old backups[29]. pgBackRest is specifically engineered to handle these lock constraints gracefully; it logs a warning regarding the undeletable object and continues its expiration run without aborting the broader backup process[29].

5.2 SFTP Implementation and libssh2 Mechanisms

For remote storage over SSH, pgBackRest utilizes the libssh2 library within its storage drivers. The software strictly enforces SSH host key validation to prevent man-in-the-middle attacks,

configurable via the repo-sftp-host-key-hash-type parameter.

Host Key Check Policy	Security Behavior
strict	The default security posture. pgBackRest refuses connections to hosts whose keys are missing or changed, forcing manual intervention to update the known_hosts file[15].
accept-new	Automatically trusts and appends new host keys to the known_hosts file upon first connection, but strictly rejects connections if an existing key changes[15].
fingerprint	Bypasses standard files and validates the host directly against a user-provided fingerprint string defined in the repo-sftp-host-fingerprint option[15].
none	Disables all cryptographic host validation. Highly discouraged for production systems[15].

Maintaining long-running SFTP sessions introduces significant engineering challenges. In extensive backups, the read-only SFTP session utilized for manifest checks can sit idle while the parallel write sessions push data. Remote servers (such as Hetzner Storage Boxes) frequently drop these idle connections, returning generic socket errors. The libssh2 implementation in src/storage/sftp/read.c occasionally suppresses non-standard errors like LIBSSH2_ERROR_SOCKET_RECV (-43), causing the system to surface ambiguous BAD_USE (-39) failures during the finalization phase of the backup[30]. The maintainers continuously optimize these storage objects to force active session reuse and implement retry logic to navigate aggressive firewall session timeouts[30].

6. Enterprise Integrations and High Availability

The resilience and low overhead of pgBackRest have made it the underlying standard for complex enterprise architectures, particularly within Kubernetes environments and automated High Availability (HA) clusters.

6.1 Kubernetes Operator Integration

Cloud-native deployments routinely utilize specialized operators, such as the Crunchy Data PostgreSQL Operator and the Percona Operator for PostgreSQL, to orchestrate database containers[23]. These operators leverage pgBackRest as their exclusive backup and recovery

engine[23].
In a typical Kubernetes topology, pgBackRest configuration is injected directly into pods via projected ConfigMaps and Secrets, mapping parameters like repo1-s3-key securely[33]. Operators deploy dedicated repo-host StatefulSets. These pods run pgBackRest as a standalone service, intercepting parallel WAL archive pushes from the primary PostgreSQL pods and managing the direct streaming of those files to attached Persistent Volumes (PVs) or external S3 buckets[23]. When a seamless, near-zero downtime migration is required—such as migrating from a Crunchy Data cluster to a Percona cluster—administrators leverage a pgBackRest standby. The new cluster initializes entirely by streaming a baseline restoration from the existing pgBackRest repository and continuously replaying archived WAL files until it achieves synchronization for a seamless failover[31].

6.2 Patroni and Distributed Consensus

In distributed environments governed by Patroni—a template for PostgreSQL High Availability using Distributed Configuration Stores (DCS)—pgBackRest serves a critical role beyond standard disaster recovery[34].
When a replica node suffers a catastrophic failure, it must be rebuilt. By default, Patroni executes pg_basebackup, which places heavy sequential read I/O pressure on the primary database leader node[34]. For multi-terabyte databases, this load degradation is unacceptable. By configuring Patroni's create_replica_methods to utilize pgBackRest, Patroni intercepts the rebuild request and commands pgBackRest to restore the failed node directly from the backup repository host[34]. This architecture offloads the entire I/O rebuild penalty onto the dedicated backup infrastructure, protecting the primary database's performance during cluster healing[34].

7. Security Architecture and Best Practices

A defining principle of the pgBackRest architecture is that security, encryption, and data integrity are handled transparently at the core protocol and storage layers, minimizing human error.

7.1 Data At Rest Encryption and pg_tde Integration

All data resting in the repository can be symmetrically encrypted natively via the repo-cipher-type=aes-256-cbc and repo-cipher-pass configuration options[35]. The tool utilizes a PBKDF2 (Password-Based Key Derivation Function 2) within its internal cipherBlock implementations (src/crypto/cipher.c), securely deriving cryptographic keys to ensure that a fully compromised backup storage volume yields no accessible data to attackers[28].
For environments requiring localized Transparent Data Encryption (TDE) directly on the PostgreSQL database, pgBackRest operates cleanly alongside extensions like pg_tde. Using ALTER SYSTEM SET shared_preload_libraries = 'pg_tde', administrators load external key management interfaces (such as Global Key Providers linked to external KMS servers) into memory[38]. Because pg_tde encrypts the physical PostgreSQL pages and WAL files before they are read by pgBackRest, the backup software will unknowingly archive encrypted data[38]. In this specific topology, administrators must explicitly disable the --archive-header-check and

--page-header-check parameters, as pgBackRest's native integrity scanners will encounter opaque, encrypted headers rather than standard PostgreSQL formats, causing false validation failures[5].

7.2 Data Integrity Verification

Backup integrity in pgBackRest is not delegated to underlying filesystem error codes. Cryptographic checksums are calculated for every file as it is copied into the repository, and these checksums are re-validated dynamically during restore or administrative verify operations[6].

Crucially, if the PostgreSQL cluster operates with data page checksums enabled, pgBackRest leverages the --checksum-page feature. As raw data streams through the I/O filters, pgBackRest natively unpacks the PostgreSQL block format in memory and validates the individual page headers in real-time[5]. If the scanner detects torn pages or silent disk corruption, the backup continues to preserve as much data as possible, but aggressive warnings detailing the exact corrupted page numbers are injected into the logs[6]. This feature allows database administrators to detect localized hardware-level corruption early, long before the clean baseline backups expire[6].

8. Conclusion

The architectural execution of pgBackRest demonstrates a masterclass in highly specialized, resilient systems engineering. By migrating entirely from Perl to C and instituting the hierarchical MemContext and setjmp/longjmp frameworks, the application achieves the throughput of bare-metal computing with the stringent memory safety of managed languages. Its deep integration with PostgreSQL internals—evidenced by block-level incrementals, in-stream page checksum validation, and precision-timed manifest synchronization—places it well beyond the capabilities of generalized file synchronization utilities. When properly configured with asynchronous archiving, bundled object storage, and TLS-secured transport protocols, pgBackRest provides a highly resilient, cryptographically secure foundation for enterprise-grade database disaster recovery.

Works cited

1. Running pgbackrest on FreeBSD - Luca Ferrari, https://fluca1978.github.io/2020/06/12/pgbackrestOnFreeBSD.html
2. introduction to pgBackRest - pgstef's blog, https://pgstef.github.io/2018/01/04/introduction_to_pgbackrest.html
3. A Long Story about how I dug into the PostgreSQL source code to write my own WAL receiver, and what came out of it, https://dev.to/alzhi_f93e67fa45b972/a-long-story-about-how-i-dug-into-the-postgresql-source-code-to-write-my-own-wal-receiver-and-what-1648
4. pgBackRest: Enterprise PostgreSQL Backup and Recovery - JusDB, https://www.jusdb.com/blog/pgbackrest-postgresql-backup-recovery
5. Configuration Reference - pgBackRest, https://pgbackrest.org/configuration.html

6. pgBackRest - Reliable PostgreSQL Backup & Restore, https://pgbackrest.org/
7. Support Alternate Compression Library · Issue #596 · pgbackrest, https://github.com/pgbackrest/pgbackrest/issues/596
8. pgbackrest/src/common/memContext.h at main - GitHub, https://github.com/pgbackrest/pgbackrest/blob/main/src/common/memContext.h
9. pgbackrest/src/common/error/error.h at main - GitHub, https://github.com/pgbackrest/pgbackrest/blob/main/src/common/error/error.h
10. ERROR: [039]: client state is 'data-get' but expected 'idle' · Issue #2744 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/2744
11. backup is failing with "invalid type at" error · Issue #1419 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/1419
12. Error `[039] "client state is 'response' but expected 'idle'"` during differential backup · Issue #2729 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/2729
13. "ERROR: [101]: raised from remote-0 ssh protocol on 'db-pgprod01': NULL result required to complete request" · Issue #2243 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/2243
14. ERROR: [029]: raised from local-2 protocol: invalid xml · Issue #1638, https://github.com/pgbackrest/pgbackrest/issues/1638
15. Command Reference - pgBackRest, https://pgbackrest.org/command.html
16. pgBackRest User Guide - Debian & Ubuntu, https://pgbackrest.org/user-guide.html
17. pgBackRest User Guide - Debian & Ubuntu / PostgreSQL 9.4, https://pgbackrest.org/1/user-guide.html
18. Backing up PostgreSQL databases with pgBackRest to Object Storage - Scaleway, https://www.scaleway.com/en/docs/tutorials/backup-postgresql-pgbackrest-s3/
19. Improve docs re: S3 best practices around repo-block and repo-bundle #2520 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/2520
20. Differential backup uses much more space than expected, wrong config? #2448 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/2448
21. pgbackrest for large databases · Issue #1595 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/1595
22. Is it safe to use noatime mount option when using pgBackRest? · Issue #1654 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/1654
23. PostgreSQL Operator Architecture & Backup and Recovery | by Himadri Talukder - Medium, https://medium.com/@himadricuet/crunchy-postgresql-operator-architecture-backup-and-recovery-fe7ef4f20af9
24. pgbackrest/src/command/restore/restore.c at main - GitHub, https://github.com/pgbackrest/pgbackrest/blob/main/src/command/restore/restore.c
25. archive-push async questions · Issue #2506 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/2506
26. How to size the parameter archive-push-queue-max ? · Issue #1195 - GitHub,

https://github.com/pgbackrest/pgbackrest/issues/1195
27. spool directory is required for a restore even if async-archive is disabled #1656 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/1656
28. loadable library and perl binaries are mismatched · Issue #687 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/687
29. pgBackRest setup for a monolith, early-stage CRM · Issue #2781 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/2781
30. SFTP repo: backup fails at stop/finalize with libssh2 when reading/listing archive metadata · Issue #2739 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/2739
31. pgBackRest - Personal blog of Yzmir Ramirez, https://rimzy.net/category/pgbackrest/
32. Migrate from Crunchy Data PostgreSQL Operator to Percona PostgreSQL Operator: Standby Cluster Method, https://www.percona.com/blog/migrate-from-crunchy-data-to-percona-postgresql-operator-standby-cluster/
33. PostgresCluster - Crunchy Data Customer Portal, https://access.crunchydata.com/documentation/postgres-operator/latest/references/crd/5.0.x/postgrescluster
34. Rebuild Patroni Replica Using pgBackRest - Percona, https://www.percona.com/blog/rebuild-patroni-replica-using-pgbackrest/
35. pgBackRest Releases - Crunchy Data Customer Portal, https://access.crunchydata.com/documentation/pgbackrest/latest/pdf/backrest.pdf
36. Error on backup: "manifest validation failed: repo size must be > 0" · Issue #2358 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/2358
37. Unable to find wal archive file in s3 repository during restore using archive-get command · Issue #1051 - GitHub, https://github.com/pgbackrest/pgbackrest/issues/1051
38. Running pgBackRest with pg_tde: A Practical Percona Walkthrough, https://percona.community/blog/2026/03/10/running-pgbackrest-with-pg_tde-a-practical-percona-walkthrough/